



Sreyas Institute of Engineering and Technology

An Autonomous Institution

Approved by AICTE, Affiliated to JNTUH

Accredited by NAAC – A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified

A
PROJECT
On

ADVANCED SUDOKU GAME USING JAVA

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
BY

JONNALAGADDA SRINIVASA SASIDHAR	23VE1A0593
ERUKONDA ABHIRAM	23VE1A0584
SANAM YASHASWINI	23VE1A05C2
DANTHOORI BHAVANI	24VE5A0509

ACADEMIC BATCH : 2023 – 2027

Content

- Abstract
- Introduction
- Architecture
- Use Case Diagram
- Class Diagram
- Activity Diagram
- Test Cases
- Output Screenshots
- Conclusion

Abstract

This presentation outlines the development of a **Sudoku Game GUI in Java**. The project delivers an interactive and engaging user experience, featuring dynamic puzzle generation, multiple difficulty levels (Easy, Medium, Hard), and real-time validation of user input. Key functionalities include a **hint system** for assistance, a **solution display** for learning or checking, and a **timer** to track performance. The game ensures a clean and intuitive interface, providing visual feedback and robust error handling for a seamless gameplay experience. This project demonstrates proficiency in Java Swing for GUI development and core algorithm design for puzzle generation and validation.

Introduction

- **Project Overview:** A fully functional Sudoku game implemented as a Graphical User Interface (GUI) application using Java Swing.
- **Objective:** To provide an intuitive and engaging platform for users to play Sudoku, incorporating features for both new players and seasoned enthusiasts.
- **Core Functionality:** Users can generate new puzzles, select various difficulty levels, input numbers, receive hints, and check their solutions.
- **User-Centric Design:** Emphasizes clear visual feedback, easy navigation, and robust input validation to ensure a smooth and enjoyable gameplay experience.
- **Foundational Skills:** Demonstrates practical application of object-oriented programming (OOP) principles, event handling, and algorithm design for puzzle logic.

Architecture

Model-View-Controller (MVC) Pattern (Conceptual):

- While not strictly separated into distinct classes for Model, View, and Controller, the application conceptually adheres to MVC principles.
- Model: ‘board’ (game state), ‘solution’ (correct answers) arrays.
- View: ‘JFrame’, ‘JPanel’, ‘JTextField’ components forming the Sudoku grid and control panel.
- Controller: Action Listeners for buttons, Document Listeners for text fields, and methods like ‘validateInput’, ‘generateSudoku’, ‘showHint’ handle user interaction and game logic.

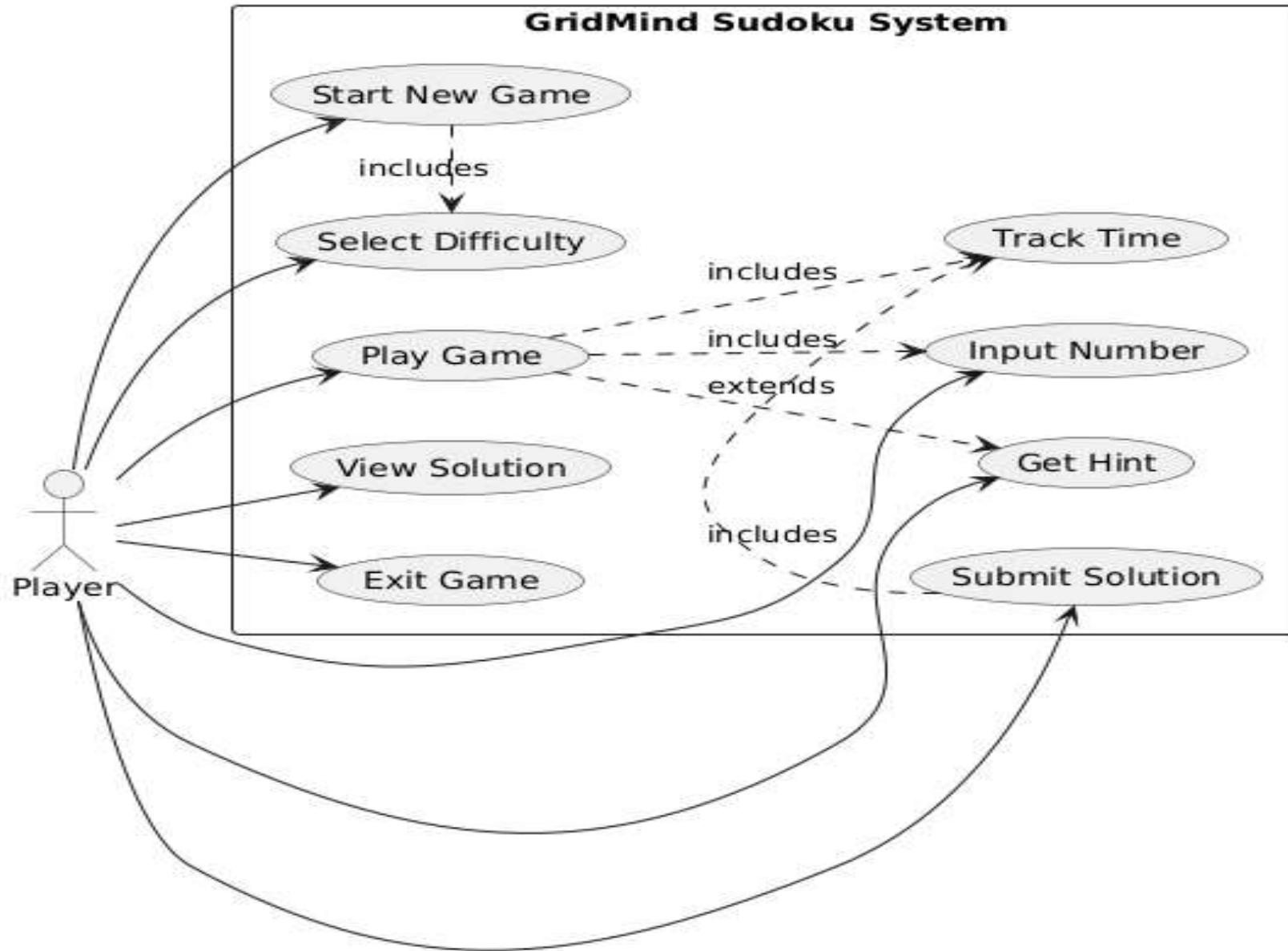
Java Swing Framework:

- Utilizes core Swing components for building the graphical user interface.
- Components include ‘JFrame’ for the main window, ‘JPanel’ for layout, ‘JTextField’ for cells, ‘JButton’ for controls, and ‘JLabel’ for the timer.

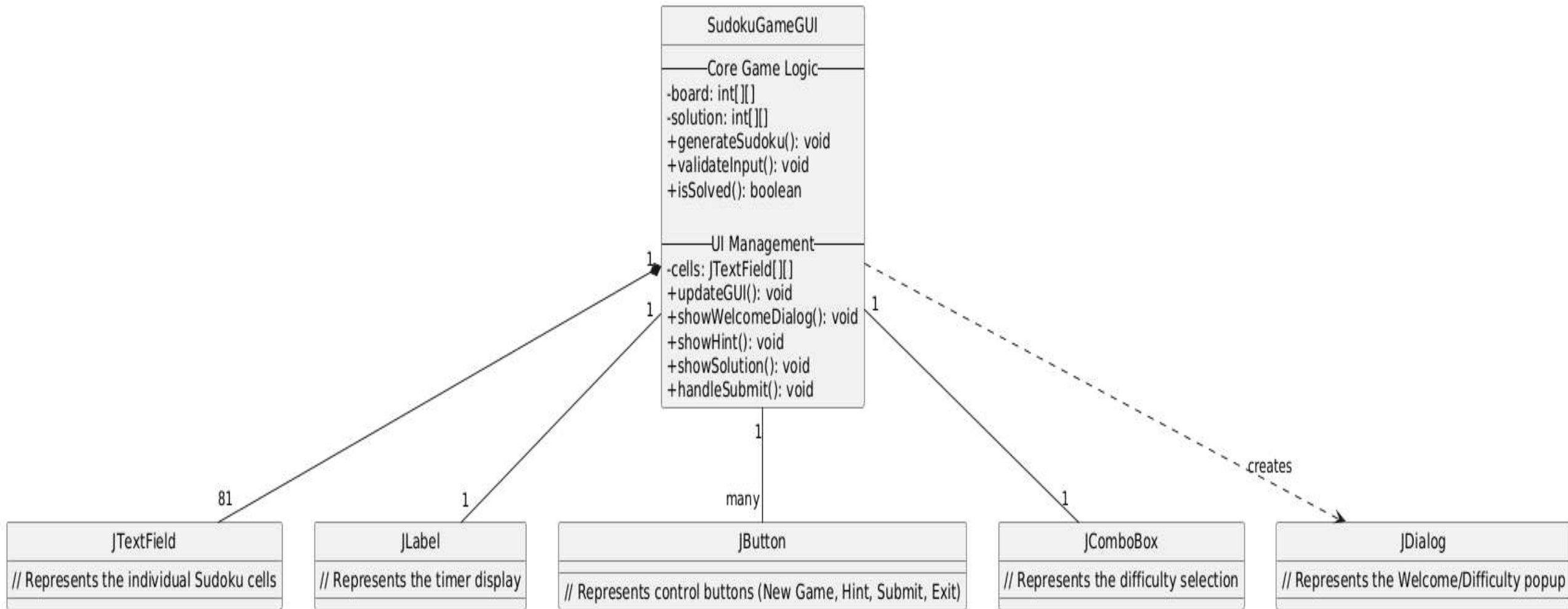
Grid Layout Management:

- Employs ‘BorderLayout’ for the main window, ‘GridLayout’ for the Sudoku grid's 3x3 boxes and cells, and ‘FlowLayout’ for the control panel.
- This layered approach ensures a well-organized and responsive UI across different screen sizes.

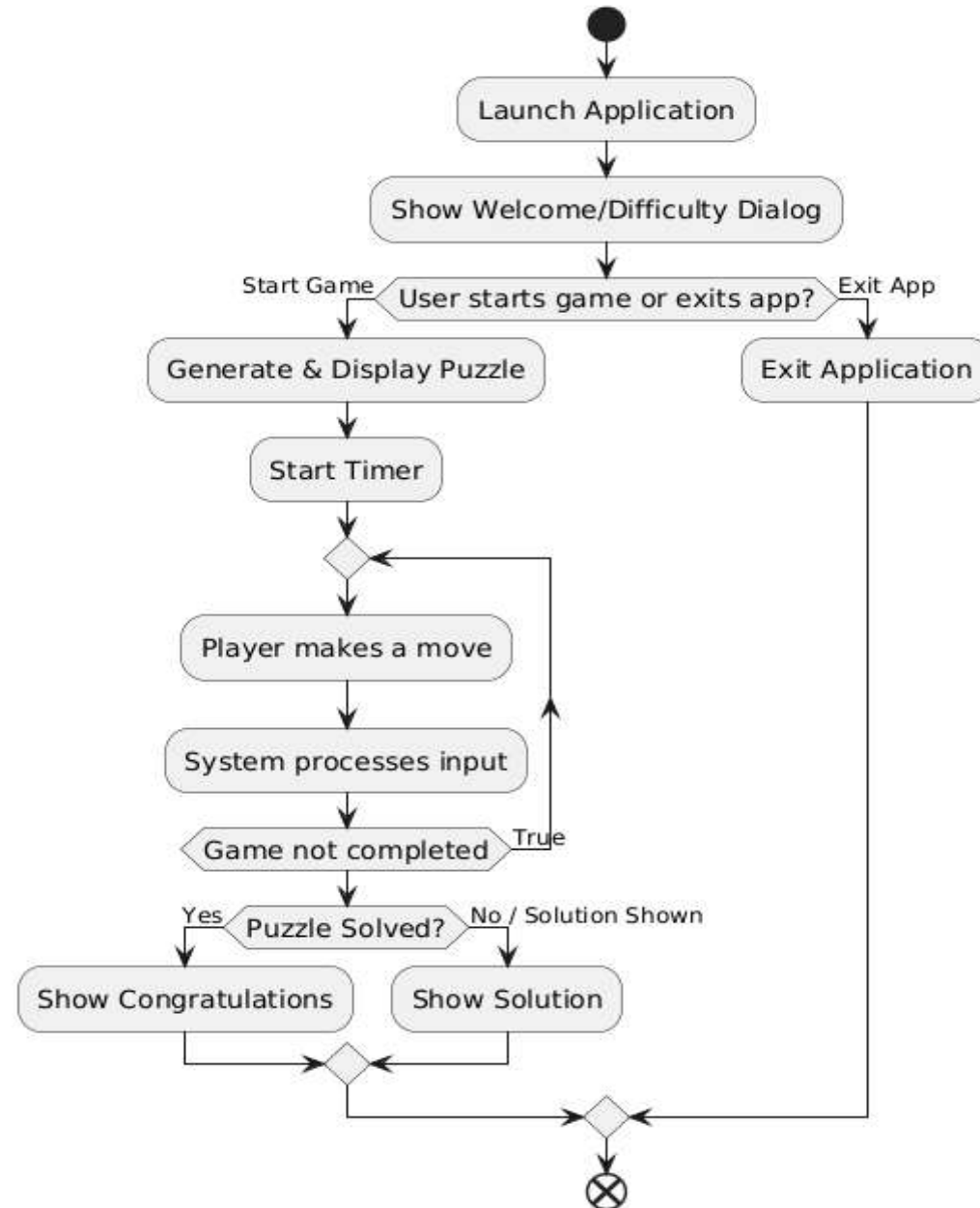
Use Case Diagram



Class Diagram



Activity Diagram



Test Cases

New Game Flow:

- **Action:** Launch / Click "New Game", select difficulty, "Start Game".
- **Outcome:** New puzzle loads, timer starts, empty cells are white and editable.
- **Purpose:** Verify game setup.

Input Validation:

- **Action:** Enter numbers (valid/invalid, numeric/non-numeric).
- **Outcome:** Valid numbers appear black, invalid ones highlight in red/yellow with tooltip; non-numeric inputs are blocked.
- **Purpose:** Ensure correct data entry and immediate feedback.

Hint Functionality:

- **Action:** Focus empty cell, click "Hint".
- **Outcome:** Correct number appears in green, cell becomes non-editable.

- **Purpose:** Confirm hint delivery.

Submit Solution:

- **Action:** Click "Submit" after filling cells.
- **Outcome:** "Congratulations" (if correct, timer stops) or "Incorrect/Incomplete" message.
- **Purpose:** Validate puzzle completion.

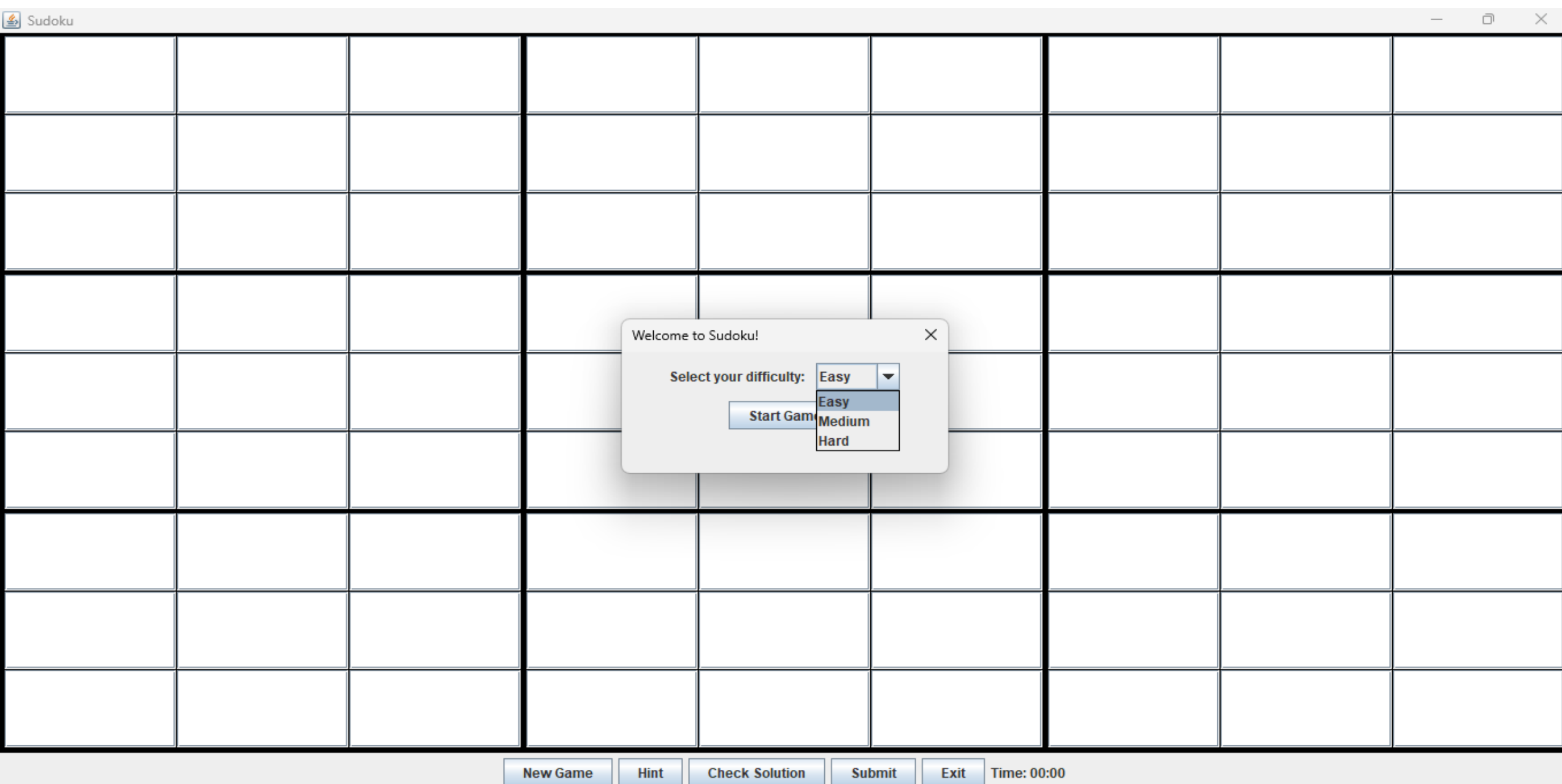
Check Solution:

- **Action:** Click "Check Solution".
- **Outcome:** Grid fills with blue/green solution, timer stops, "Solution displayed" message.
- **Purpose:** Verify forced solution display.

Exit Application:

Click "Exit", confirm. Ensure with user confirmation.

Output Screenshots



1		8	1	2	3	4	6	
7	6	3	4		5	1	2	1
	4	1		9	6	3	5	8
1	8	5		7		2	4	
3		6	5	4	8	9		1
	7		3		2	6		
6	3	2	8	5	1	7	9	4
5		4		3	7	8		6
8	1		9		4	5	3	

9	5	8	1	2	3	4	6	7
7	6	3	4	8	5	1	2	9
2	4	1	7	9	6	3	5	8
1	8	5	6	7	9	2	4	3
3	2	6	5	8	9	7	1	
4	7	9	3	1	2	6	8	5
6	3	2	8	5	1	7	9	4
5	9	4	2	3	7	8	1	6
8	1	7	9	6	4	5	3	2

Sudoku Solved



The solution has been displayed!

OK

New Game

Hint

Check Solution

Submit

Exit

Time: 01:01

4	7			5	2	3	9	
9			4	7	6	1	8	
		6		8	9	2	4	
1		5		3			7	4
6		4	9		7	5	3	1
	9			1	4	6		8
3	5	9		4		8		2
2	4			6	3	7		9
	6	7				4		3

Exit Sudoku



Are you sure you want to exit?

Yes

No

New Game

Hint

Check Solution

Submit

Exit

Time: 00:05

Conclusion

- **Successful Implementation:** The Sudoku project successfully delivers a complete and interactive Sudoku game with a user-friendly GUI.
- **Key Achievements:**
 - Dynamic puzzle generation with adjustable difficulty.
 - Real-time input validation and helpful visual feedback.
 - Integrated timer for tracking gameplay.
 - User-friendly features: hint system, solution display, and clear submission process.
 - Enhanced user experience with welcome dialog, full-screen mode, and confirmed exit.
- **Learning Outcomes:** This project reinforced principles of Java Swing for GUI design, event handling, backtracking algorithms for puzzle generation, and effective state management in a game application.